

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Алимов Р.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создает дочерний процесс. Первой строкой пользователь вводит имя файла, которое передается дочернему процессу. Родительский и дочерний процессы представлены разными программами. Родительский процесс передает введенные пользователем команды через неименованный канал, подключенный к стандартному потоку ввода дочернего процесса. Дочерний процесс получает строки вида «число число число<newline>», вычисляет сумму чисел типа float и записывает результат каждой строки в указанный файл. Количество чисел в строке может быть произвольным.

Общий метод и алгоритм решения

Программа состоит из двух исполняемых файлов: parent.out и child.out. Родительский процесс считывает имя файла, создает неименованный канал pipe() и дочерний процесс fork(). В дочернем процессе читающий конец канала перенаправляется в стандартный ввод с помощью dup2(), после чего запускается child.out через execl(). Родитель передает введенные пользователем данные в канал, закрывает его после EOF и ожидает завершения дочернего процесса.

Дочерняя программа открывает указанный файл, читает данные из стандартного ввода блоками по 1024 байта, преобразует числа функцией strtod() и суммирует их. При конце строки сумма записывается в файл. Для обработки данных, переходящих через границы блоков чтения, используется фиксированный буфер; ошибки системных вызовов проверяются.

Использованные системные вызовы

- pipe(int pipefd[2]) — создает неименованный канал; pipefd[0] используется для чтения, pipefd[1] — для записи.
- fork(void) — создает дочерний процесс. После вызова оба процесса продолжают выполнение с одной точки, но получают разные возвращаемые значения.
- dup2(int oldfd, int newfd) — переназначает файловый дескриптор; используется для подключения читающего конца канала к стандартному вводу child.
- execl(...) — заменяет образ текущего процесса программой child.out, сохраняя сам процесс и его PID.
- read(int fd, void *buf, size_t count) — считывает данные из стандартного ввода или канала.
- write(int fd, const void *buf, size_t count) — передает данные в канал и записывает вычисленные результаты в файл.
- open(const char *pathname, int flags, mode_t mode) — открывает или создает результирующий файл для записи.
- close(int fd) — закрывает ненужные файловые дескрипторы и обеспечивает корректное формирование EOF в канале.
- wait(...) — ожидает завершения дочернего процесса и предотвращает появление zombie-процесса.
- exit(...) — завершает дочерний процесс при ошибке после fork().

Код программы

parent.c

```
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#define BUFFER_SIZE 1024
```

```

int main(void) {
    char filename[BUFFER_SIZE];
    int pos = 0;
    char c;

    write(STDOUT_FILENO, "Enter filename: ", 16);

    while (pos < BUFFER_SIZE - 1) {
        ssize_t n = read(STDIN_FILENO, &c, 1);

        if (n == -1) {
            write(STDERR_FILENO, "read error\n", 11);
            return EXIT_FAILURE;
        }

        if (n == 0 || c == '\n')
            break;

        filename[pos++] = c;
    }

    filename[pos] = '\0';

    if (pos == 0) {
        write(STDERR_FILENO, "empty filename\n", 15);
        return EXIT_FAILURE;
    }

    int pipe_fd[2];

    if (pipe(pipe_fd) == -1) {
        write(STDERR_FILENO, "pipe error\n", 11);
        return EXIT_FAILURE;
    }

    pid_t pid = fork();

    if (pid == -1) {
        write(STDERR_FILENO, "fork error\n", 11);
        close(pipe_fd[0]);
        close(pipe_fd[1]);
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        close(pipe_fd[1]);

        if (dup2(pipe_fd[0], STDIN_FILENO) == -1) {
            write(STDERR_FILENO, "dup2 error\n", 11);
            close(pipe_fd[0]);
            exit(EXIT_FAILURE);
        }

        close(pipe_fd[0]);

        execl("./child.out", "child.out", filename, (char *)NULL);
        write(STDERR_FILENO, "exec error\n", 11);
        exit(EXIT_FAILURE);
    } else {
        close(pipe_fd[0]);

        char buffer[BUFFER_SIZE];
        ssize_t bytes_read;

        while ((bytes_read = read(STDIN_FILENO, buffer, BUFFER_SIZE)) > 0) {
            ssize_t total_written = 0;

```

```

        while (total_written < bytes_read) {
            ssize_t bytes_written = write(
                pipe_fd[1],
                buffer + total_written,
                bytes_read - total_written
            );

            if (bytes_written == -1) {
                write(STDERR_FILENO, "write error\n", 12);
                close(pipe_fd[1]);
                wait(NULL);
                return EXIT_FAILURE;
            }

            total_written += bytes_written;
        }

        if (bytes_read == -1) {
            write(STDERR_FILENO, "read error\n", 11);
            close(pipe_fd[1]);
            wait(NULL);
            return EXIT_FAILURE;
        }

        close(pipe_fd[1]);

        if (wait(NULL) == -1) {
            write(STDERR_FILENO, "wait error\n", 11);
            return EXIT_FAILURE;
        }
    }

    return EXIT_SUCCESS;
}

```

child.c

```

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BLOCK_SIZE 1024
#define BUFFER_SIZE 2048

int main(int argc, char *argv[]) {
    if (argc != 2) {
        write(STDERR_FILENO, "wrong arguments\n", 16);
        return EXIT_FAILURE;
    }

    const char *filename = argv[1];

    int file_fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (file_fd == -1) {
        write(STDERR_FILENO, "open error\n", 11);
        return EXIT_FAILURE;
    }

    char block[BLOCK_SIZE];
    char ring[BUFFER_SIZE];
    char number[BLOCK_SIZE + 1];

    size_t number_len = 0;
    size_t head = 0;
    size_t tail = 0;

```

```

size_t count = 0;
ssize_t bytes_read;
float sum = 0.0f;

while ((bytes_read = read(STDIN_FILENO, block, BLOCK_SIZE)) > 0) {
    for (ssize_t i = 0; i < bytes_read; ++i) {
        if (count == BUFFER_SIZE) {
            write(STDERR_FILENO, "buffer overflow\n", 16);
            close(file_fd);
            return EXIT_FAILURE;
        }

        ring[tail] = block[i];
        tail = (tail + 1) % BUFFER_SIZE;
        ++count;
    }

    while (count > 0) {
        char c = ring[head];
        head = (head + 1) % BUFFER_SIZE;
        --count;

        if (c == ' ' || c == '\t' || c == '\n') {
            if (number_len > 0) {
                number[number_len] = '\0';
                char *end;
                float value = strtod(number, &end);

                if (end == number || *end != '\0') {
                    write(STDERR_FILENO, "invalid number\n", 15);
                    close(file_fd);
                    return EXIT_FAILURE;
                }

                sum += value;
                number_len = 0;
            }

            if (c == '\n') {
                char output[64];
                int len = snprintf(output, sizeof(output), "%f\n", sum);

                if (len < 0) {
                    close(file_fd);
                    return EXIT_FAILURE;
                }

                if (write(file_fd, output, (size_t)len) == -1) {
                    write(STDERR_FILENO, "write error\n", 12);
                    close(file_fd);
                    return EXIT_FAILURE;
                }

                sum = 0.0f;
            }
        } else {
            if (number_len == BLOCK_SIZE) {
                write(STDERR_FILENO, "number too long\n", 16);
                close(file_fd);
                return EXIT_FAILURE;
            }

            number[number_len++] = c;
        }
    }
}

if (bytes_read == -1) {
    write(STDERR_FILENO, "read error\n", 11);
}

```

```

        close(file_fd);
        return EXIT_FAILURE;
    }

    if (number_len > 0) {
        number[number_len] = '\0';
        char *end;
        float value = strtod(number, &end);

        if (end == number || *end != '\0') {
            write(STDERR_FILENO, "invalid number\n", 15);
            close(file_fd);
            return EXIT_FAILURE;
        }

        sum += value;

        char output[64];
        int len = snprintf(output, sizeof(output), "%f\n", sum);

        if (len < 0) {
            close(file_fd);
            return EXIT_FAILURE;
        }

        if (write(file_fd, output, (size_t)len) == -1) {
            write(STDERR_FILENO, "write error\n", 12);
            close(file_fd);
            return EXIT_FAILURE;
        }
    }

    close(file_fd);
    return EXIT_SUCCESS;
}

```

Протокол работы программы

Перед тестированием программа собирается командой `make`. Завершение ввода выполняется сочетанием `Ctrl-D`, после чего родительский процесс закрывает записывающий конец канала и ожидает дочерний процесс.

Тест 1. Суммирование дробных и отрицательных чисел

```

$ ./parent.out
Enter filename: res.txt
1.5 2.5 10
4.2 0.8
100 -20.5
^D
$ cat res.txt
14.000000
5.000000
79.500000

```

Тест 2. Отрицательные числа и несколько пробелов между значениями

```

$ ./parent.out
Enter filename: res2.txt
-10 -20 5.5
1      2.5      3
^D
$ cat res2.txt
-24.500000
6.500000

```

Тест 3. Строка длиной более одного блока чтения

Для проверки обработки границ блоков формируется строка, содержащая 700 единиц. Ее длина превышает 1024 байта, поэтому данные гарантированно обрабатываются несколькими чтениями.

```
$ { echo "res3.txt"; python3 -c 'print("1 " * 700)'; } | ./parent.out
$ cat res3.txt
700.000000
```

Все тесты завершились корректно. Программа обрабатывает отрицательные и дробные значения, произвольное количество чисел в строке и входные данные, размер которых превышает размер одного блока `read()`. Для очень больших по модулю значений возможна потеря точности, обусловленная ограниченной точностью типа `float`.

Вывод

В ходе лабораторной работы были получены практические навыки создания процессов и организации межпроцессного взаимодействия в Linux с помощью неименованных каналов. Было реализовано перенаправление стандартного ввода дочернего процесса через `dup2()` и запуск отдельной программы с помощью `exec`. Основные сложности были связаны с правильным закрытием концов канала, обработкой потока данных без привязки к границам `read()` и учетом возможной частичной записи. Реализованная программа выполняет требования варианта 2 и корректно записывает суммы входных значений типа `float` в указанный файл.